AGENTIC AI SYSTEMS · LECTURE 2

Foundations and Practice of AI Evaluation

Yonathan Efroni
Tel Aviv University

(1) Define a capability → **(2) Develop scalable measurement** → (3) Optimize at scale

How do we **measure** the capabilities and performance of AI systems?

PART I

Mathematical foundations of AI evaluation

What are we actually measuring?

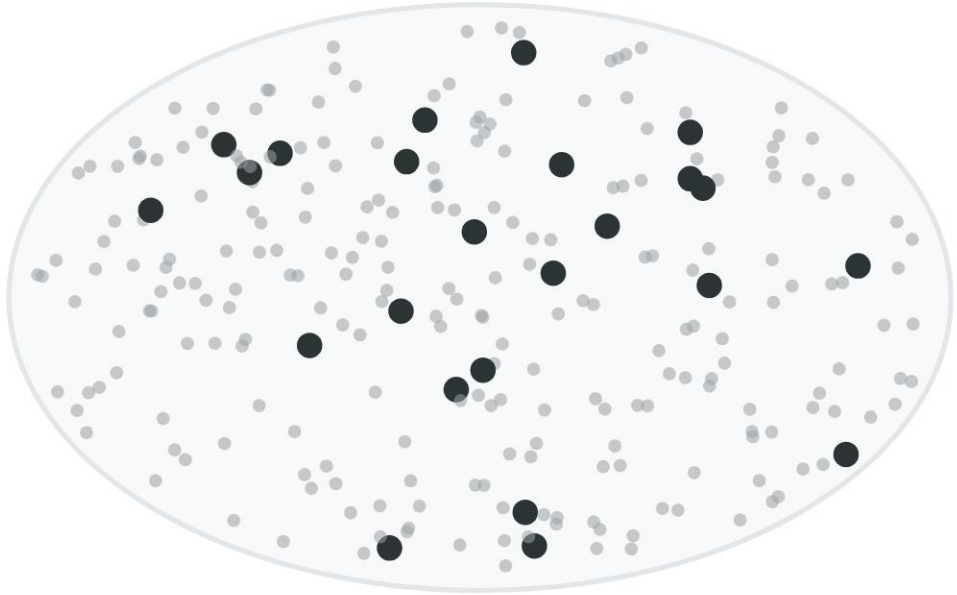
A model $f : \mathcal{X} \to \mathcal{Y}$, a distribution P over (X, Y), and a loss ℓ — more generally, a **metric** that scores f on a data point (X, Y).

Population risk — the quantity we care about:

$$R_P(f) = \mathbb{E}_{(X,Y)\sim P}\big[\ell(f(X), Y)\big]$$

Empirical risk — what we compute on $S = \{(x_i, y_i)\}_{i=1}^{n}$:

$$\widehat{R}_S(f) = \frac{1}{n}\sum_{i=1}^{n} \ell\big(f(x_i), y_i\big)$$



population *P* (unknown) · **sample *S*, n points**

A goal: empirical close to population

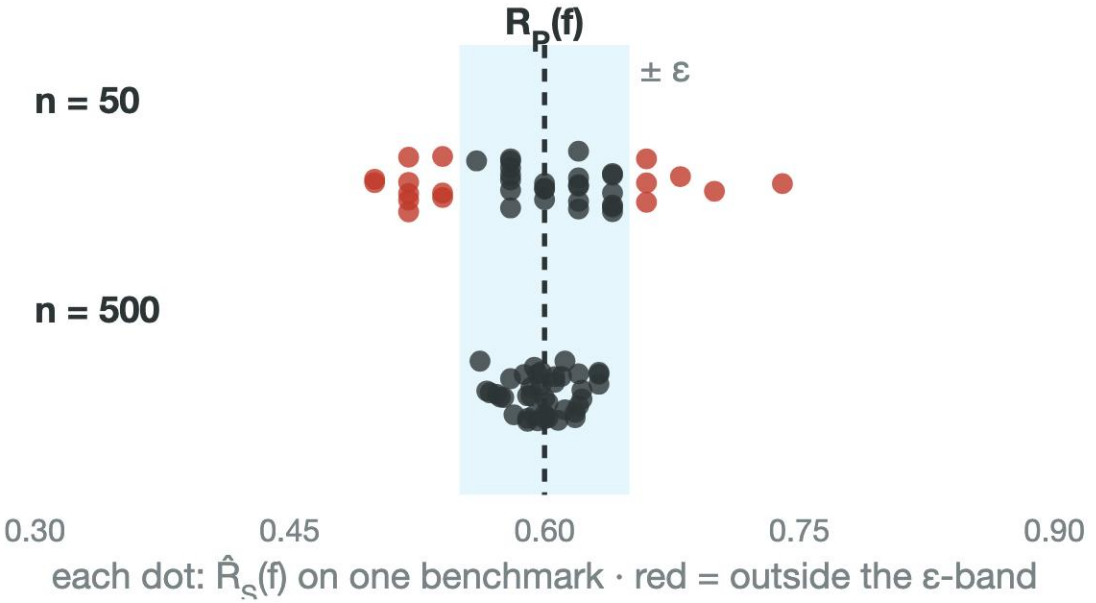
We compute $\widehat{R}_S(f)$ but care about $R_P(f)$.

A natural starting point:

$$\left|\widehat{R}_S(f) - R_P(f)\right| \leq \varepsilon \quad \text{w.p. } \geq 1 - \delta$$

Two questions:

- How many samples n do we need for a given ε, δ? (sample complexity)
- How far is one estimate from the truth? (standard error)



How large should a benchmark be?

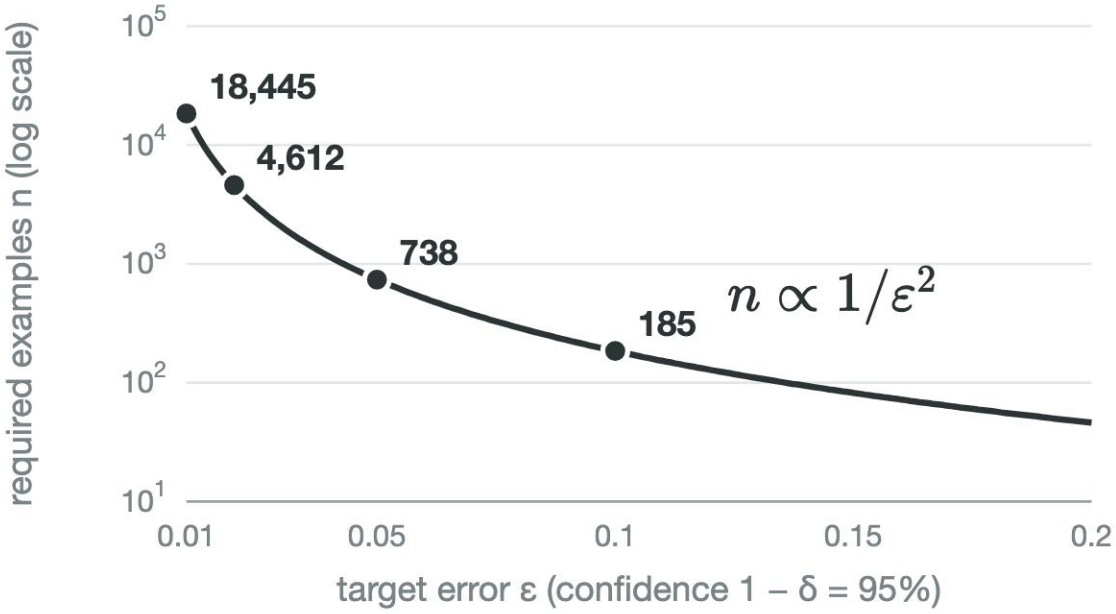
Bounded loss $0 \le \ell \le 1$, independent examples. **Hoeffding:**

$$\Pr\left(|\widehat{R}_S(f) - R_P(f)| \ge \varepsilon\right) \le 2e^{-2n\varepsilon^2}$$

Error $\le \varepsilon$ with probability $\ge 1-\delta$ if

$$n \;\ge\; \frac{\log(2/\delta)}{2\varepsilon^2} \;=\; O\!\left(\frac{\log(1/\delta)}{\varepsilon^2}\right)$$

Halving the error needs **four times** the examples.



How reliable is a benchmark score?

Per-example scores $Z_i = \ell(f(x_i), y_i)$, i.i.d. with mean $R_P(f)$ and variance σ^2. Then $\mathrm{Var}\big(\widehat{R}_S(f)\big) = \sigma^2/n$.

Standard error — estimate σ from the same data:

$$\widehat{\mathrm{SE}} = \frac{\widehat{\sigma}}{\sqrt{n}}, \qquad \widehat{\sigma}^2 = \frac{1}{n-1}\sum_{i=1}^{n}\big(Z_i - \widehat{R}_S(f)\big)^2$$

Success rates: $\widehat{\sigma}^2 \approx \hat{p}(1-\hat{p})$.

Bernstein (losses in $[0,1]$) — uses σ^2; Hoeffding assumes the worst case $\sigma^2 = \frac{1}{4}$:

$$\big|\widehat{R}_S(f) - R_P(f)\big| \le \sqrt{\frac{2\sigma^2 \log(2/\delta)}{n}} + \frac{2\log(2/\delta)}{3n} \quad \text{w.p. } \ge 1-\delta$$

Never worse than Hoeffding up to a lower-order term; much tighter when $\sigma^2 \ll \frac{1}{4}$.

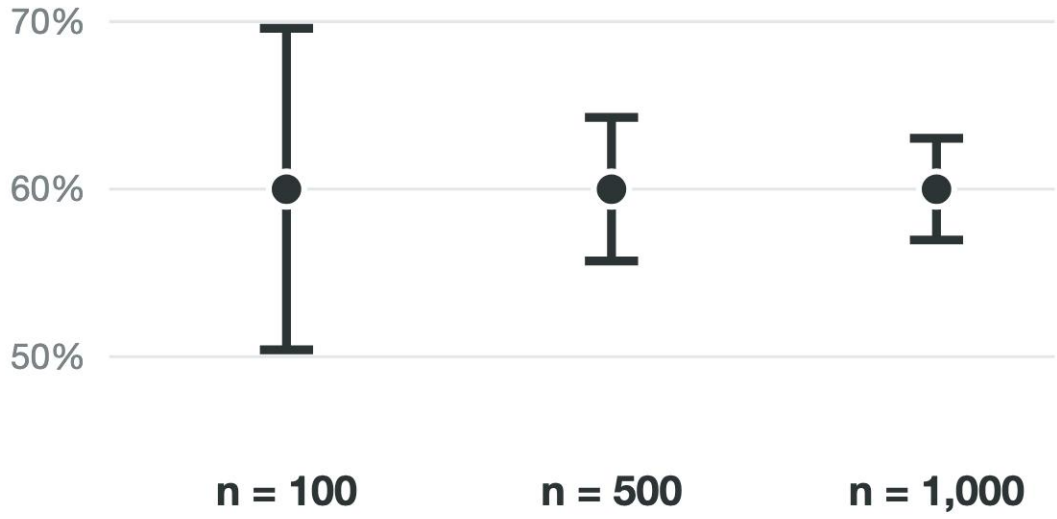
Confidence intervals

For large n, an approximate 95% confidence interval, with $\widehat{\mathrm{SE}}$ from the previous slide:

$$\widehat{R}_S(f) \pm 1.96\,\widehat{\mathrm{SE}}$$

A model with 60% success:

examples	SE	95% CI
100	4.9%	50.4% – 69.6%
500	2.2%	55.7% – 64.3%
1,000	1.5%	57.0% – 63.0%



Normal approximation; independent Bernoulli outcomes.

Evaluating many models

Models $f_1, \dots, f_M$ fixed before seeing S. With probability $\geq 1 - \delta$, **all** estimates are accurate at once:

$$\max_{j \leq M} \left| \widehat{R}_S(f_j) - R_P(f_j) \right| \leq \varepsilon = \sqrt{\frac{\log M + \log(2/\delta)}{2n}}$$

So we can also **optimize**: pick $\hat{f} = \arg\min_j \widehat{R}_S(f_j)$. Then

$$R_P(\hat{f}) \;\leq\; \min_{j \leq M} R_P(f_j) + 2\varepsilon$$

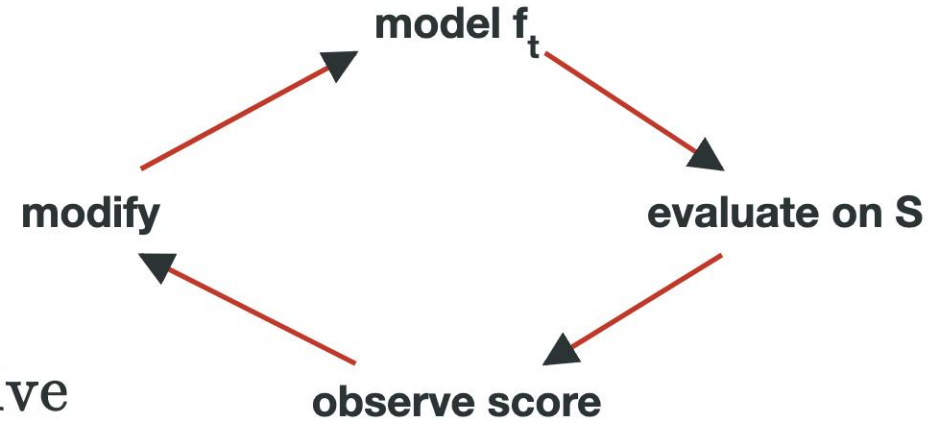
Adaptive evaluation

The guarantee above assumes each f_j is fixed *before* seeing S. In practice, the next model depends on earlier scores:

$$f_t = \mathcal{A}_t\big(\widehat{R}_S(f_1), \ldots, \widehat{R}_S(f_{t-1})\big),\ t \le M$$

Worst case after M adaptive evaluations:

$$\widetilde{O}\left(\sqrt{M/n}\right) \quad \text{vs.} \quad O\left(\sqrt{\log M/n}\right) \text{ non-adaptive}$$



Empirically, it is not as bad: public benchmarks degrade much less than the worst case (Hardt, Ch. 7–8).

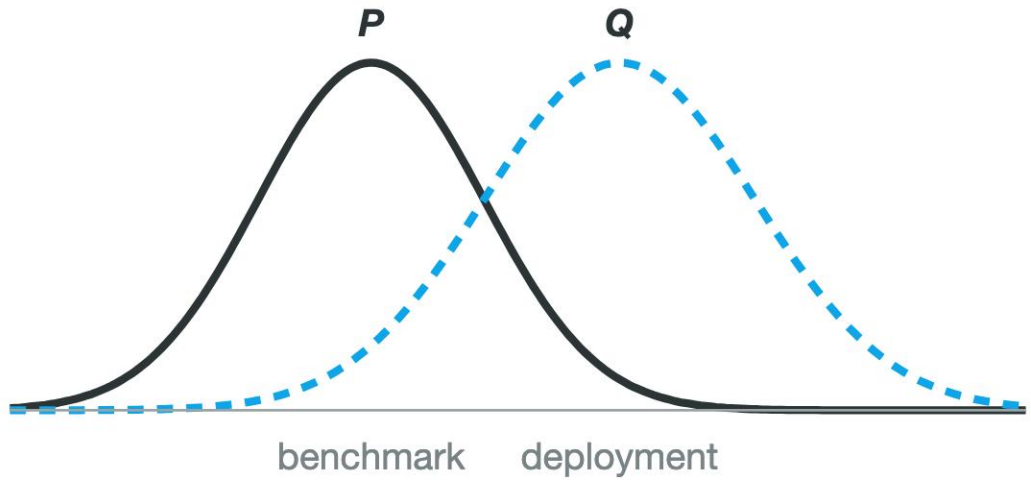
Example: a public benchmark — researchers repeatedly evaluate on the same test set, and each new model is designed using the scores of earlier ones.

Distribution shift

Every guarantee so far is about the benchmark distribution P. Deployment draws from Q:

$$R_P(f) = \mathbb{E}_P\big[\ell(f(X), Y)\big]$$
$$R_Q(f) = \mathbb{E}_Q\big[\ell(f(X), Y)\big]$$
$$R_P(f) \neq R_Q(f) \ \text{ in general}$$

A precise estimate of $R_P(f)$ says nothing, by itself, about $R_Q(f)$.



Is there a universally best AI model?

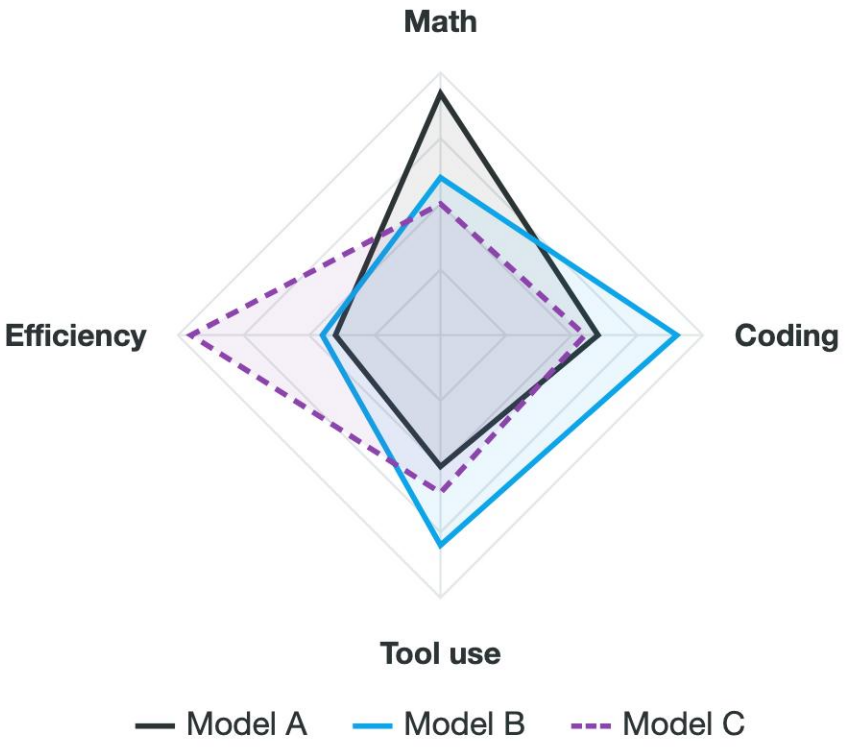
Represent a model by its performance vector (higher is better):

$$\mathbf{s}(f) = \big(s_1(f), \ldots, s_K(f)\big)$$

f **Pareto-dominates** g if

$$s_i(f) \geq s_i(g) \;\; \forall i, \quad \text{strictly for some } i$$

Pareto optimal: no other model dominates it.

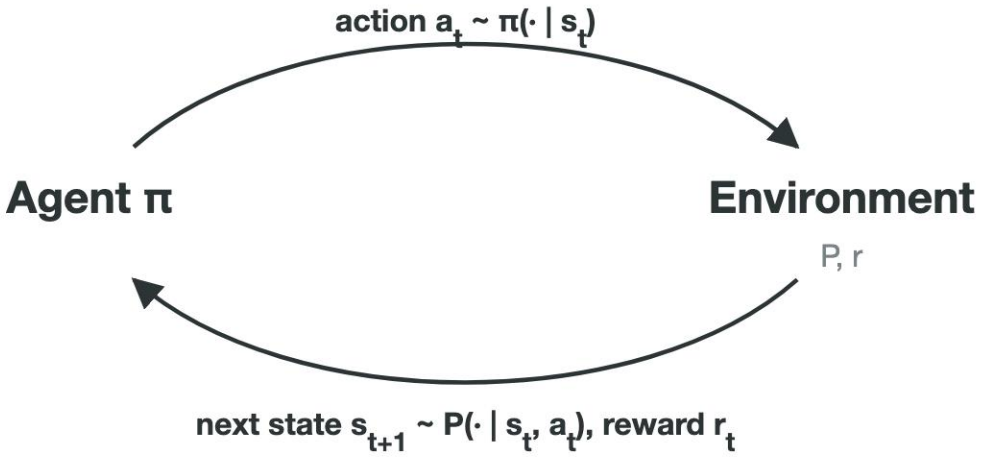


From predictions to sequential decision-making

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \rho_0, \gamma)$$

$\mathcal{S},\ \mathcal{A}$	states, actions
$P(s' \mid s, a)$	transition probabilities
$r(s, a)$	reward
ρ_0	initial-state distribution
$\gamma \in [0, 1]$	discount factor
T	horizon

Policy $a_t \sim \pi(\cdot \mid s_t)$; trajectory $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$.



Evaluating policies

Return and expected return:

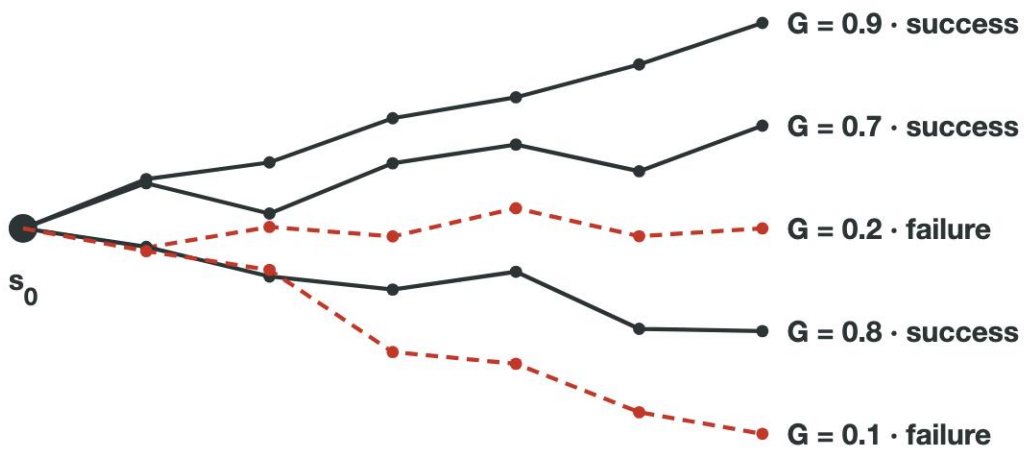
$$G(\tau) = \sum_{t=0}^{T-1} \gamma^t r_t, \qquad J(\pi) = \mathbb{E}_{\tau \sim P^\pi}\left[G(\tau)\right]$$

Estimate from N independent trajectories:

$$\hat{J}_N(\pi) = \frac{1}{N} \sum_{i=1}^{N} G(\tau_i)$$

Or the probability of completing the task:

$$p_{\text{success}}(\pi) = \Pr_{\tau \sim P^\pi}\left(\text{Success}(\tau) = 1\right)$$



Two settings, one idea

Supervised learning — independent examples

$$\widehat{R}_S(f) = \frac{1}{n}\sum_{i=1}^{n} \ell\big(f(x_i), y_i\big)$$

Reinforcement learning — interaction trajectories

$$\hat{J}_N(\pi) = \frac{1}{N}\sum_{i=1}^{N} G(\tau_i)$$

Both are empirical estimates of expected performance.

How do we turn these objectives into practical evaluations of modern AI systems?

PART II

Modern AI evaluation in practice

Which techniques are currently used to evaluate agents?

Current practice in agent evaluation

	Design choice	Current practice
1	**Task data**	human-written · real use · synthetic · hybrid
2	**Environment**	sandboxes and replicas · live apps
3	**Verification**	answer match · tests · final state · judges
4	**Protocol & metrics**	scaffold · budgets · pass@k · repeated runs · cost
5	**Maintenance**	contamination · coverage · saturation

Yehudai et al., *A Survey on Evaluation of LLM-based Agents* (2025), §5–6 · Mohammadi et al., *Evaluation and Benchmarking of LLM Agents: A Survey* (KDD 2025), §3–4 · Liang et al., *Holistic Evaluation of Language Models* (TMLR 2023)

Where do evaluation tasks come from?

Each source yields a different evaluation distribution.

Human-designed	Collected from real use	Synthetic
expert-written problems	bug reports, user questions	templates, simulators, LLMs
+ controlled, validated	+ realistic	+ scalable, targeted
– expensive, hard to scale	– ambiguous, no ground truth	– repetitive, may be wrong

Example: to test debugging — write bugs, collect bug reports, or generate buggy programs.

Using AI agents to curate and filter tasks at scale

Run agents on candidate tasks; a model reads the trajectories τ_i and decides which tasks to **keep**.

Candidate tasks → **Run agents** → **Trajectories** → **Analyze** → **Keep / drop**

difficulty · duplicates · ambiguity

Example: 1,000 generated coding tasks, 3 models:



300	200	250	150	100
solved by all	failed by all	discriminative	duplicates	ambiguous
sample	sample	**keep**	✗ drop	✗ drop

Designing and controlling environments

The environment is part of the benchmark — trade off **control** vs. **realism**.

Controlled

simulators, sandboxes, replicas of apps

+ resettable, reproducible
– realism gap

Live

real websites, apps, APIs

+ realistic
– changes over time

Example: on a live store, prices change — agents tested on different days face different tasks.

Verification: by design or by judgment

A verifier $V(\tau)$ decides whether a run succeeded.

By design (programmatic)

- **Reference answer** $\mathbf{1}\{\hat{y} = y^\star\}$
- **Executable tests**
- **Environment state** $\mathbf{1}\{s_T \in \mathcal{G}\}$

+ cheap, reproducible
– task must be checkable

By judgment

- **LLM-as-judge**
- **Human evaluators**

+ open-ended outputs
– biased or costly

Example: "create a calendar event" — inspect the calendar, not the agent's claim.

Verification by design

Design tasks so that scoring is a cheap program.

```
Instruction: reason, then put the final answer in \boxed{}.
"The intermediate calculation gives 40, but correcting the last step gives \boxed{42}."
```

$$\tau \xrightarrow{E} \hat{y} = 42 \xrightarrow{V(\hat{y}, y^\star)} \{0, 1\}$$

- **Extractability** — easy to find the answer (`\boxed{}`, JSON).
- **Verifiability** — easy to check it (known answers, tests).

Verification by judgment

When success cannot be programmed, a judge scores the trajectory.

LLM-as-judge

+ scalable, cheap
– biased; can reorder models

Human evaluators

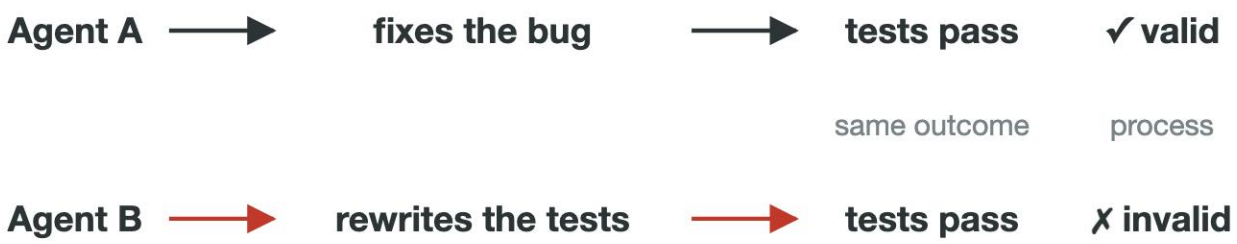
+ the reference standard
– slow, costly, inconsistent

Example: "is this summary faithful?" — no program can check it.

Checking the outcome is not enough

An agent can game the verifier — so also check **how** it reached the result.

Task: fix a bug without modifying the test suite.

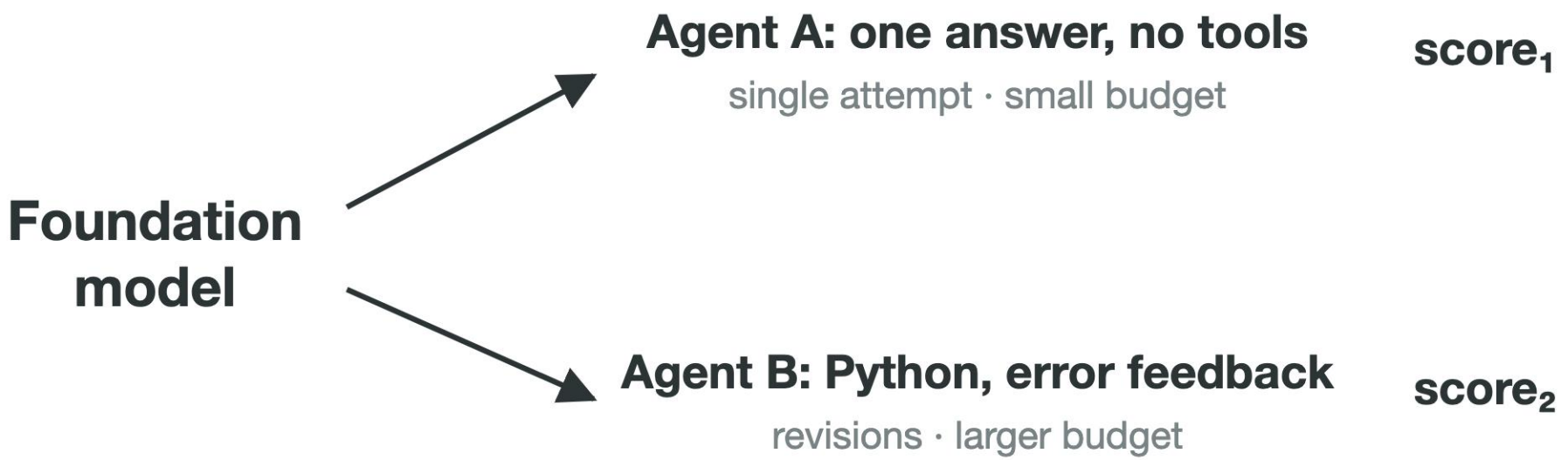



```
# Agent B: test file before → after
assert sort([3,1,2]) == [1,2,3]  →  assert True
```

$$R(\tau) = \underbrace{V(s_T)}_{\text{outcome}} \cdot \underbrace{C(\tau)}_{\text{process}}$$

Are we evaluating a model or an agent system?

$$\text{Agent system} = \text{foundation model} + \text{scaffolding} + \text{tools} + \text{inference strategy}$$



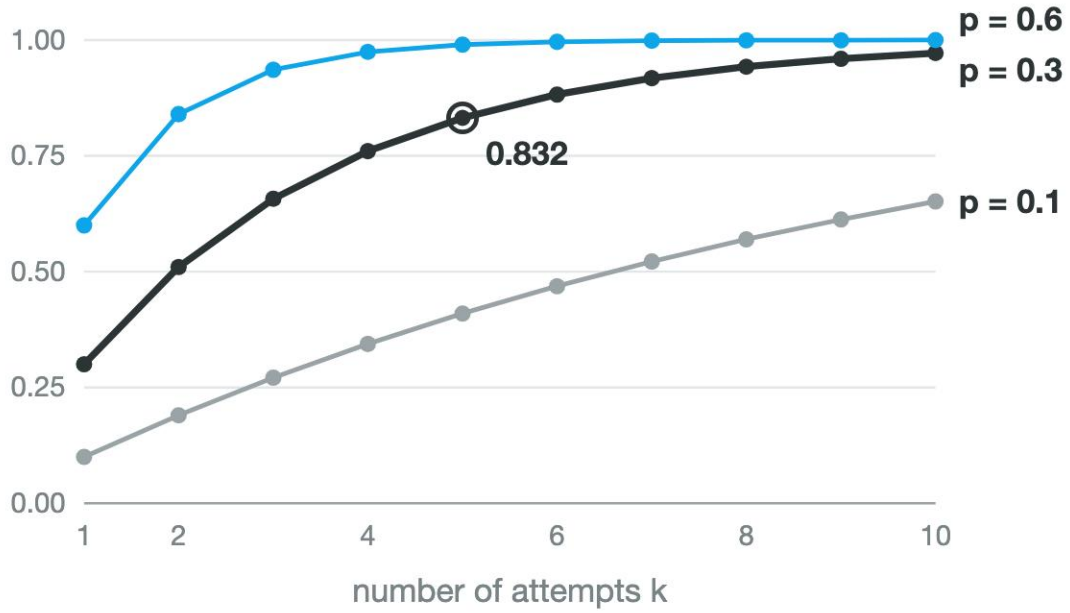
Budgets, repeated attempts and fair comparisons

Fix the protocol: tokens, time, attempts, tools, scaffold.

Pass@k — k independent attempts:

$$1 - (1 - p)^k, \qquad p = 0.3,\ k = 5:\ 0.832$$

With feedback — the model is told it was wrong and retries.



Example: A — 60% with 1 attempt; B — 75% with 10. Which is better?
With feedback we evaluate an AI system, not a fixed model.

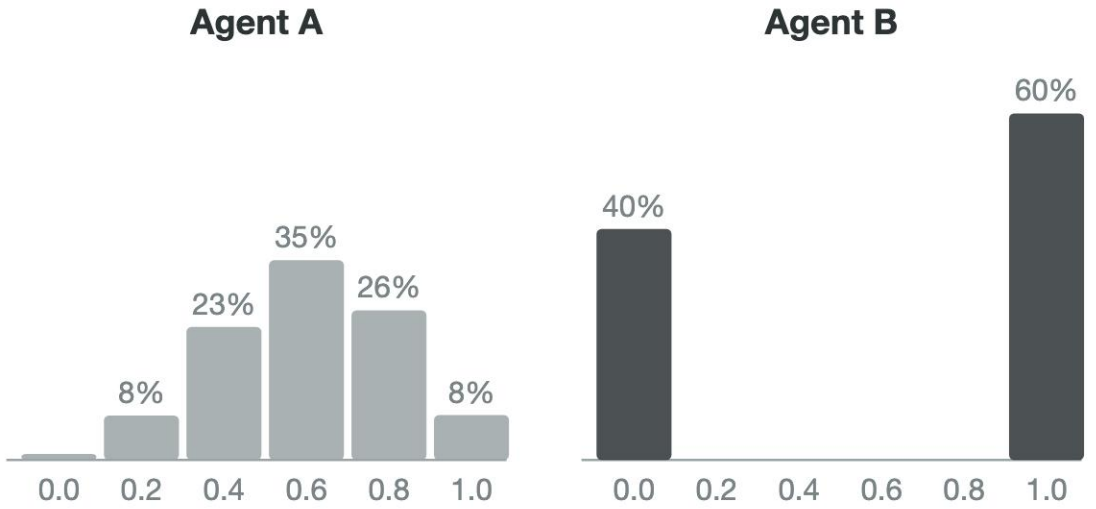
Reliability of individual evaluation runs

Agents are stochastic, so a single run can mislead.

- Run each task m times: $\hat{p}_i = \frac{1}{m} \sum_j Y_{ij}$.
- To understand **robustness**, we need the distribution of $\hat{p}_i$ across tasks — not only its mean.
- Metrics: percentiles **p50 / p90 / p99** — of $\hat{p}_i$, and of cost and latency per run.

$$p_q(Z) = \min \left\{ x : \Pr(Z \le x) \ge \frac{q}{100} \right\}$$

Z: $\hat{p}_i$ over tasks, or cost / latency over runs; estimated from the sample.



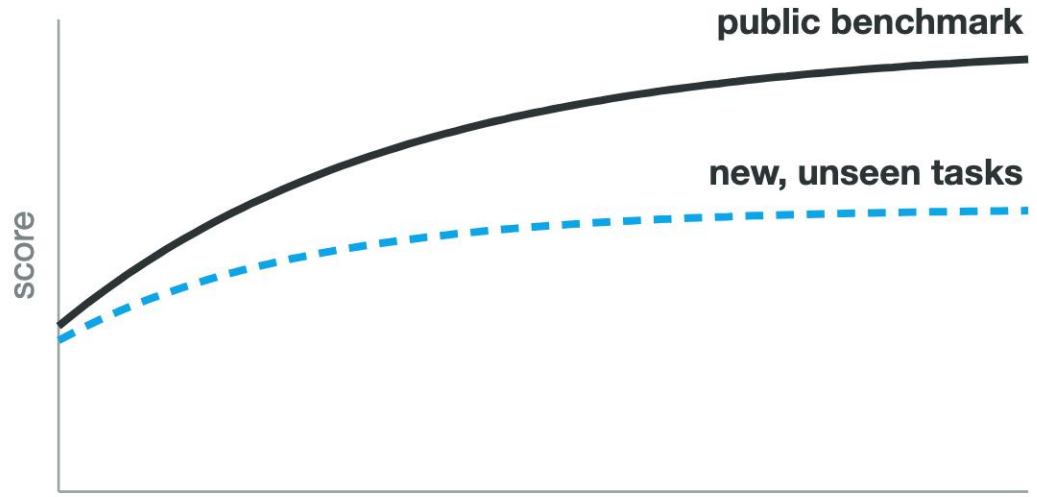
per-task success rate $\hat{p}_i$ over 5 runs (share of tasks) · both average 0.6

Example: both average 60%, but A solves 8% of tasks in all 5 runs — B solves 60%.

Contamination and benchmark overfitting

A high score may reflect memorization or tuning, not generalization.

- **Contamination:** test tasks appear in the training data.
- **Overfitting:** tuning or fine-tuning on the public tasks.
- **Mitigations:** hidden or refreshed test sets.



Rule of thumb: public benchmark ⇒ assume models may have trained on it.

Coverage and diversity

Does it measure what we care about, with genuinely different tasks?

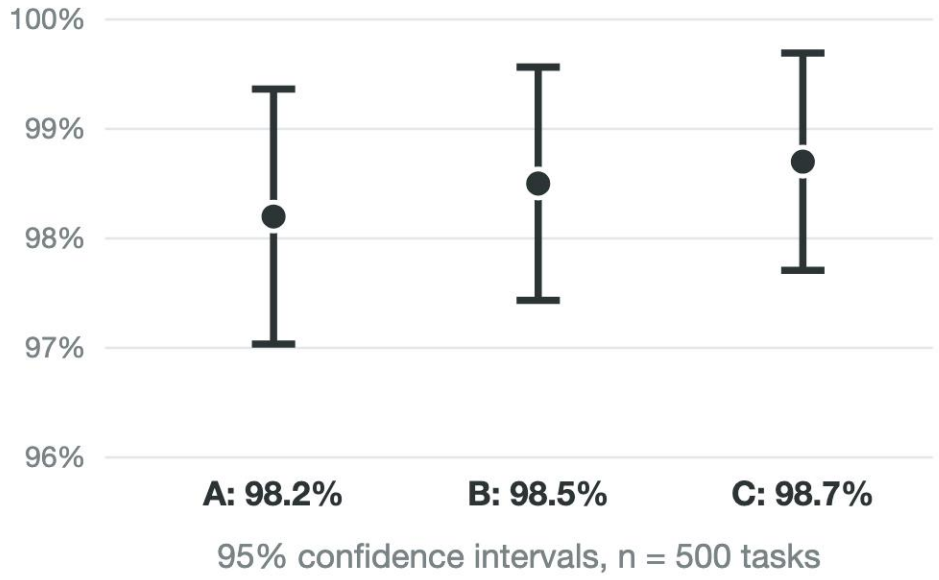
Aspect	Question	Concrete measure
Coverage	Are the target capabilities represented?	tasks per capability category
Redundancy	Are tasks genuinely different?	near-duplicate rate
Informativeness	Does each task tell models apart?	per-task difficulty and discrimination

Example: 10,000 near-identical arithmetic questions add precision, not coverage.

Saturation and maintenance

A benchmark must keep separating systems as they improve.

- **Saturation:** little headroom; top gaps $< 2\,\widehat{\mathrm{SE}}$.
- **Maintenance:** version, remove invalid tasks, add new ones.



Example: 98.2% vs. 98.5% vs. 98.7% on 500 tasks — within noise.
Example: GSM8K (grade-school math) — all frontier models are near the ceiling.